УДК 004.4'422

А.М. Балышев (2 курс бакалавриата),
Д.С. Косарев (инженер-исследователь)

РАЗРАБОТКА КОМПИЛЯТОРА ДЛЯ ЗАЩИТЫ ОТ АТАК НА ЦЕПОЧКИ ПОСТАВОК

Атакой на цепочку поставок программного обеспечения (англ. software supply chain attack) называют ситуацию, при которой злоумышленник получает доступ к программной системе путём внедрения вредоносного кода в компонент, используемый организацией при разработке или развертывании этой системы. Ещё в 1984 году Кен Томпсон в лекции "Reflections on Trusting Trust" описал атаку, при которой вредоносный код внедряется компилятором в генерируемые им программы, что делает аудит исходного кода компилируемых программ недостаточным для выявления уязвимостей [1].

Одним из наиболее распространенных подходов к борьбе с атаками на цепочки поставок является Diverse Double-Compiling [2]. Метод состоит в том, чтобы проверять воспроизводимость артефактов при сборке несколькими независимыми компиляторами. Если удаётся получить побитово идентичные артефакты, то вероятность того, что сборка была скомпрометирована, крайне мала, поскольку злоумышленнику пришлось бы компрометировать несколько независимых сборок идентичным образом. Подобный подход широко используется в проектах с открытым исходным кодом: большое количество пользователей могут независимо собирать проекты из публично доступных исходных файлов и сверять результаты. Одной из основных проблем этого подхода является высокая сложность достижения детерминированности сборок. Окружение сборки требуется явно описывать и полностью повторять: архитектура процессора, версия операционной системы и версии зависимостей должны совпадать, иначе результаты не будут воспроизводимы, даже если сборка не была скомпрометирована. Известно, что даже наиболее популярные языковые экосистемы имеют проблемы с обеспечением воспроизводимости сборок без дополнительной конфигурации и/или внесения исправлений в их реализацию [3].

Rukaml [4] — проект по разработке компилятора функционального языка, развиваемый на математико-механическом факультете СПбГУ, преимущественно в рамках семестровых учебных практик. Rukaml написан на OCaml и позволяет порождать RISC-V 64 ASM, AMD64 ASM и LLVM IR. Система сборки dune, используемая в Rukaml, уделяет большое внимание воспроизводимости сборок, что достигается с помощью механизмов фиксации зависимостей [5], изоляции сборки в песочнице [6] и вероятностной проверки воспроизводимости [7]. В рамках работы предлагается обеспечить возможность собирать с помощью Rukaml языковые инструменты и обеспечить защиту собираемых программ от атак на цепочки поставок. Ожидается, что преимущества выбранной экосистемы позволят добиться воспроизводимости сборок на уровне текстового представления программы — на уровне языков ассемблера. Текстовое представление человекочитаемо, что позволит распознавать скрытую логику путём ручной проверки результата компиляции. Это полезно в случае, если проверка воспроизводимости артефактов провалилась, чтобы проанализировать расхождения результатов и зафиксировать попытку атаки, а также в качестве дополнительной гарантии безопасности. Кроме того, в практике компиляторной инженерии известны эффективные оптимизации объема генерируемого компилятором кода [8], применение которых существенно упрощает процесс аудита.

Важно понимать, что такой подход требует наличия доверенного транслятора языка ассемблера, что является отдельной инженерной задачей, остающейся за рамками работы над Rukaml.

Целью работы в рамках проекта Rukaml является реализация поддержки языковых конструкций, необходимых для разработки компиляторов и других языковых инструментов, а также обеспечение защиты собираемых программ от атак на цепочки поставок.

Для достижения этой цели необходимо решение *задач*, перечисленных ниже.

1. Реализация поддержки алгебраических типов данных и сопоставления выражений с образцом.
2. Расширение стандартной библиотеки примитивами форматированного вывода, работы с файловой системой и обработки аргументов командной строки.
3. Оптимизация размера порождаемого компилятором кода.
4. Обеспечение воспроизводимости сборок.

Реализация решения даст пользователю возможность описывать и обрабатывать промежуточные представления программ такие, как деревья абстрактного синтаксиса и компилировать программы, работающие в режиме утилиты командной строки. Достижение детерминированности сборок позволит осуществлять проверку воспроизводимости артефактов. Выбранный подход — компактное текстовое представление — позволит проводить ручное выявление скрытой логики. Это даст возможность собирать языковые инструменты в среде, защищенной от атак на цепочки поставок. На момент написания заявки решение *находится в разработке*.

ЛИТЕРАТУРА

1. Ken Thompson. "Reflections on Trusting Trust", 1984
2. David A. Wheeler et al. "Fully Countering Trusting Trust through Diverse Double-Compiling", 2009
3. Giacomo Benedetti et al. "An Empirical Study on Reproducible Packaging in Open-Source Ecosystems", 2025
4. Rukaml. [Электронный ресурс] Режим доступа: https://github.com/Kakadu/rukaml/blob/master/README.md (дата обращения: 17.03.2026)
5. Dune Locking. [Электронный ресурс] Режим доступа: https://dune.readthedocs.io/en/latest/tutorials/dune-package-management/locking.html (дата обращения: 15.03.2026)
6. Dune Sandboxing. [Электронный ресурс] Режим доступа: https://dune.readthedocs.io/en/stable/concepts/sandboxing.html (дата обращения: 15.03.2026)
7. Dune Reproducibility Check. [Электронный ресурс] Режим доступа: https://dune.readthedocs.io/en/stable/caching.html#reproducibility(дата обращения: 15.03.2026)
8. Intel, Methods to Optimize Code Size. [Электронный ресурс] Режим доступа: https://www.intel.com/content/www/us/en/docs/dpcpp-cpp-compiler/developer-guide-reference/2025-1/methods-to-optimize-code-size.html (дата обращения: 15.03.2026)